# Talk-to-Data Delivery Blueprint — Phase 5: Prototype / MVP Build

Daniel Brule · linkedin.com/in/danielbrule
v1.0 · June 2026

1 Executive summary
2 Phase overview
  2.1 Objective
  2.2 Scope of the phase
  2.3 What this phase does not do
  2.4 Expected duration and level of effort
  2.5 Main participants and decision owners
3 Readiness decision and delivery implications
  3.1 Possible Phase 5 outcomes
  3.2 Minimum conditions to proceed
  3.3 Common reasons to remediate, redesign or pause
  3.4 How Phase 5 shapes later phases
4 MVP build activities overview
  4.1 Activity sequence
  4.2 Activity logic
  4.3 Practitioner note
5 Core build activities
  5.1 Confirm MVP scope and build plan
  5.2 Confirm MVP governance, ownership and decision rights
  5.3 Set up environment, CI/CD and test harness
  5.4 Connect approved data, metadata and tools
  5.5 Implement metadata retrieval and grounding
  5.6 Implement AI/model routing and clarification
  5.7 Implement query generation, validation and execution
  5.8 Implement result interpretation and answer generation
  5.9 Build lightweight UI and feedback capture
  5.10 Tune, test and package evidence
6 MVP evidence decision pack
  6.1 MVP evidence pack
  6.2 Evidence pack quality test
7 Exit criteria and handover
  7.1 Required exit outputs
  7.2 Handover to later phases
  7.3 Exit decision wording
  7.4 Practitioner note

## 1 Executive summary

Phase 5 turns the Phase 4 orchestration design into a bounded working MVP. If Phase 4 defines the intended question-to-answer flow, Phase 5 tests that flow against reality.

The purpose of this phase is not to build the final production service. It is to challenge the approved design through implementation: can the system retrieve the right metadata, select the right AI and model patterns, generate constrained queries, validate them before execution, interpret results, surface caveats, handle follow-up questions, fail safely and capture enough evidence for later validation?

The central discipline is to build for evidence, not demonstration value. A polished interface is not enough. The MVP should show which parts of the design work, which assumptions fail, where AI adds value, where deterministic controls are safer than model judgement, and where the data foundation, metadata contract or orchestration design must be refined.

Phase 5 should make production-readiness gaps explicit rather than implying that a working MVP is ready for release.

The MVP should include a lightweight user interface or interaction shell, but only to expose the flow, test user-facing behaviour and capture feedback. It should not become a full product-design exercise.

The phase should also introduce the minimum operational scaffolding needed to avoid rework later: environment separation, CI/CD discipline, prompt and model versioning, basic security controls, logging, tracing, cost and latency capture, error handling, feedback capture and a simple quality dashboard. These controls do not need to be production-grade, but they should make the MVP observable, testable and supportable.

Phase 5 should also state clearly what remains unresolved for production. Formal security assurance, full evaluation, load testing, release governance, incident management, monitoring thresholds, support model, data retention, privacy review, resilience, accessibility, adoption planning and production run-cost approval should be carried into later validation, pilot and production-readiness phases rather than implied by the MVP.

By the end of Phase 5, stakeholders should be able to decide whether the MVP provides enough evidence to proceed to formal validation, whether the scope should be constrained, whether the build or orchestration design needs remediation, whether the governed foundation needs refinement, or whether the initiative should pause.

The main output is a working MVP and an evidence pack covering the implemented flow, AI/model choices, validation behaviour, logs and traces, quality and cost findings, known limitations, issue backlog and production-readiness gaps.** **

## 2 Phase overview

Phase 5 builds the bounded MVP from the approved Phase 4 orchestration design. It turns the designed question-to-answer flow into a working system that can be tested with realistic questions, controlled data, defined AI/model choices, validation checks, logging and user feedback.

The key discipline is to build an observable and testable MVP, not a polished demo. Phase 5 should make the approved flow real enough to test, but still bounded enough to remain safe, explainable and correctable.

### 2.1 Objective

The objective of Phase 5 is to build a bounded MVP and produce the evidence needed for formal validation, controlled user testing and production-readiness planning. It should confirm:

- **Implemented flow:** how the system moves from user question to metadata retrieval, AI/model routing, query generation, validation, execution, answering and feedback.
- **AI and model use:** where AI is used, which model patterns are selected, and where deterministic rules or human review are safer.
- **Data and metadata integration:** whether approved Phase 3 assets and Phase 4 metadata contracts can be consumed reliably at runtime.
- **Query and answer controls:** whether queries are bounded, validated and executed safely, and whether answers show sources, assumptions, caveats and safe-failure behaviour.
- **User interaction:** whether a lightweight interface supports realistic questions, follow-ups, clarification, refusals, answer review and feedback.
- **Operational scaffolding:** whether the MVP includes enough CI/CD, versioning, logging, tracing, basic security, cost capture, latency capture, error handling and quality monitoring to support validation and pilot use.
- **Production-readiness gaps:** what remains to be strengthened during validation, pilot and production-readiness phases.

### 2.2 Scope of the phase

Phase 5 should remain bounded to the approved users, questions, data assets, metadata sources, tools and orchestration design. It should implement enough of the system to test the intended flow without becoming a broad product build or uncontrolled expansion exercise.

In scope are the MVP application flow, AI/model calls, retrieval components, query generation, validation checks, execution hand-off, result interpretation, answer generation, lightweight UI, feedback capture, logging, tracing, cost and latency capture, basic security controls, deployment discipline and issue tracking.

The test for inclusion is simple: if an item is required to test the MVP safely, diagnose failures, support Phase 6 validation and remediation or prepare for Phase 7 user testing, it belongs in Phase 5. Issues exposed during build should be fixed, constrained, deferred or routed back rather than hidden in prompts or manual workarounds.

### 2.3 What this phase does not do

Phase 5 does not approve the system for production. A working MVP shows that the approved design can operate in a bounded way; it does not prove that the service is ready for broad release.

It also does not replace formal validation, redesign the governed foundation or deliver the full product experience. Issues with answer quality, security, access exposure, resilience, cost, scalability, Phase 3 artefacts or Phase 4 design assumptions should be tested, refined or routed into later phases rather than patched around during the build.

### 2.4 Expected duration and level of effort

The effort depends on delivery intent:

- For a POC, Phase 5 should prove that the main components can run together: approved data access, metadata retrieval, AI/model calls, query generation, validation, execution, answer generation and basic feedback.
- For an MVP, Phase 5 should produce a first usable version of the capability, with enough deployment discipline, versioning, validation, logging, cost/latency capture, feedback and quality monitoring to support Phase 6 validation and remediation and Phase 7 user testing. The phase is complete when the MVP can run the approved flow end-to-end, expose its limitations and provide enough evidence to proceed to formal validation.

### 2.5 Main participants and decision owners

Phase 5 is a build phase, but it should not be treated as an engineering-only activity. The MVP will only be useful if product, data, AI, security, evaluation and operating stakeholders help decide what is implemented, tested, constrained and carried forward.

| Role | Main responsibility in Phase 5 |
|---|---|
| Product owner | Confirms MVP scope, priorities, user-facing behaviour and trade-offs. |
| AI / solution architect | Owns the implemented orchestration flow, model/task choices, tool integration and design compromises. |
| AI / ML engineer | Builds model interactions, prompts, routing, retrieval, evaluation hooks and model/version controls. |
| Data / semantic owner | Confirms that metadata, metric definitions, caveats and approved artefacts are used correctly. |
| Analytics / data engineer | Implements data access, query execution, performance checks and data-layer integration. |
| Software engineer | Builds the application, APIs, deployment, UI and integration. |
| Security / governance lead | Reviews basic access controls, tool permissions, logging, data exposure and unresolved security gaps. |
| Evaluation owner | Defines technical tests, early answer-quality checks, issue taxonomy and evidence requirements for Phase 6. |
| Business SME | Tests questions, answers, caveats, clarifications / failure behaviour. |
| Operating owner | Reviews supportability, monitoring needs, incident implications, feedback handling and production-readiness gaps. |

The key point is accountability. Phase 5 should not end with a working MVP that no one can explain, test, secure, support or improve.

## 3 Readiness decision and delivery implications

Phase 5 should not end with a statement that the MVP “works”. It should end with a clear decision on whether the implemented system provides enough evidence to move into formal validation.

The decision should be based on the working flow, model and tool behaviour, metadata grounding, query validation, answer quality, test results, logging, cost and latency observations, known limitations, governance baseline and unresolved production-readiness gaps.

A proceed decision does not mean the system is production-ready. It means the MVP is mature enough to be tested formally in Phase 6 and, if validated, used for controlled pilot testing in Phase 7.

For a POC, Phase 5 may be the final delivery stage if the objective is structured learning and feasibility assessment. If the output will be used by real users for controlled testing, decision support or pilot activity, it should proceed through formal validation before use.

### 3.1 Possible Phase 5 outcomes

| Outcome | Meaning |
|---|---|
| Proceed to validation | The MVP is stable and evidenced enough to move into formal validation. |
| Proceed with constraints | Validation can start, but only for restricted users, questions, data, tools or answer types. |

| | |
|---|---|
| Remediate build | Implementation gaps must be fixed before validation. |
| Refine design | Phase 4 orchestration assumptions were incomplete, unsafe or impractical. |
| Refine foundation | Phase 3 assets or metadata are not usable enough at runtime. |
| Pause | Value, risk, cost, usability or ownership is not credible enough to continue. |
| Close POC | The POC has produced enough learning to stop, redesign or decide whether to invest in an MVP. |

### 3.2 Minimum conditions to proceed

Phase 5 should proceed to formal validation only when the team can confirm:

- **The approved flow works end-to-end:** realistic questions can move through retrieval, AI/model routing, query generation, validation, execution, answer generation and feedback.
- **AI use is explicit:** model choices, deterministic controls and human-review points are documented.
- **Metadata grounding works:** the system can retrieve and apply the right metrics, dimensions, joins, filters, caveats and examples.
- **Queries are controlled:** unsafe, invalid, unauthorised or excessive queries are blocked or routed for review.
- **Answers are explainable:** responses show the relevant source, assumptions, caveats and safe-failure behaviour.
- **The MVP is testable and observable:** tests, logs, traces, validation outcomes, security/access events, cost, latency, errors and feedback are captured.
- **Governance is credible:** owners, change controls, issue routes, limitations and next actions are clear enough to support Phase 6 validation.

### 3.3 Common reasons to remediate, redesign or pause

The team should avoid moving into validation where any of the following apply:

- **The flow only works in demos:** realistic questions frequently fail, drift or require manual intervention.
- **Prompt patches hide design gaps:** prompts compensate for weak metadata, unclear logic or missing controls.
- **Retrieval is unreliable:** the system selects wrong, stale or incomplete metadata.
- **Validation is too shallow:** checks catch syntax errors but miss scope, access, joins, cost or plausibility.
- **Answers overstate certainty:** caveats, assumptions, limitations or refusals are missing or weak.
- **Logs are insufficient:** failures cannot be diagnosed, reproduced or evaluated.
- **Cost or latency is already fragile:** the MVP pattern is unlikely to support validation or pilot use.
- **Ownership is unclear:** no one owns defects, model behaviour, metadata fixes, quality issues or support.

### 3.4 How Phase 5 shapes later phases

Phase 5 provides the evidence base for formal validation, pilot planning and production-readiness work. Its outputs should not be treated as a demo record; they should become the test evidence, issue backlog and delivery constraints for later phases.

| Later phase | What Phase 5 should provide |
|---|---|
| Phase 6 — Validation, assurance and remediation | Working MVP, test cases, logs, traces, validation results, quality findings, security gaps and failure examples. |
| Phase 7 — Controlled pilot | Usable MVP, supported scope, known limitations, feedback route, user-facing caveats and issue triage process. |
| Phase 8 — Production readiness and go-live | Production-readiness gaps, cost and latency evidence, support needs, governance gaps and hardening backlog. |

A strong Phase 5 does not remove the need for validation. It makes validation possible.

## 4 MVP build activities overview

Phase 5 translates the approved design into an implemented MVP. The activities are ordered from scope confirmation and

build setup through AI/model implementation, data and metadata integration, query control, answer generation, interface, testing and evidence packaging. Governance runs across the sequence: decisions, changes, risks, model choices, validation gaps and production-readiness issues should be visible, owned and traceable throughout the build.

The activities should not be treated as a rigid waterfall. Build work will normally iterate as retrieval, prompts, validation, query execution, answer quality, cost, latency and logging issues are discovered. The important point is that changes should be visible, tested and governed rather than patched informally.

### 4.1 Activity sequence

| Activity | Main question | Main output |
|---|---|---|
| 1. Confirm MVP scope and build plan | What exactly will be built, tested and excluded? | Build scope, backlog and acceptance criteria. |
| 2. Confirm MVP governance and decision rights | Who owns, validates and approves build decisions, changes, risks and evidence? | Role map, approval points, decision log and change route. |
| 3. Set up environment, CI/CD and test harness | Can the MVP be built, deployed, tested and traced safely? | Environments, deployment path, test harness and trace setup. |
| 4. Connect approved data, metadata and tools | Can the MVP access only approved assets and interfaces? | Working connections, permissions and access boundaries. |
| 5. Implement metadata retrieval and grounding | Can the system retrieve and apply the right governed context? | Retrieval component, grounding tests and retrieval issues. |
| 6. Implement AI/model routing and clarification | Which AI/model patterns are used, and when should the system clarify? | Routing, prompts, clarification rules and model choices. |
| 7. Implement query generation, validation and execution | Can questions become bounded, validated and executable queries? | Query generation, validation gates and execution hand-off. |
| 8. Implement result interpretation and answer generation | Can results become clear, caveated and explainable answers? | Answer generation, caveats and safe-failure behaviour. |
| 9. Build lightweight UI and feedback capture | Can users test the flow without a full product UI? | Interaction shell, answer display and feedback capture. |
| 10. Tune, test and package evidence | Are prompts, retrieval, model routing, validation and answers good enough for formal validation? | Test results, tuned behaviour, logs, traces, limitations, issue backlog and handover pack. |

### 4.2 Activity logic

The build should follow the approved Phase 4 design, but not assume that the design is correct. Phase 5 is where retrieval quality, model choices, validation rules, metadata structure, query cost, answer format and failure behaviour are tested against implementation reality.

The MVP should therefore be built around observable steps. The team should be able to see which metadata was retrieved, which model or rule was used, what query was generated, which validations passed or failed, what was executed, how the answer was produced and what limitations remain.

### 4.3 Practitioner note

Phase 5 should not optimise for a smooth demo. It should optimise for a system that can be tested, challenged and improved. A brittle MVP with a good interface is less valuable than a narrower MVP that exposes its assumptions, logs its behaviour and makes the next delivery decision clear.

## 5 Core build activities

### 5.1 Confirm MVP scope and build plan

**Purpose**: Confirm what Phase 5 will build, test and exclude, and turn the approved Phase 4 design into a practical MVP build plan.

**Activities**

- Confirm the approved users, questions, answer types and exclusions.
- Confirm the Phase 3 assets, metadata sources and Phase 4 orchestration components required for the build.
- Define the MVP backlog across data, metadata, AI/model use, validation, UI, logging and testing.
- Agree the evidence required before the MVP can move to Phase 6 validation.

- Identify assumptions or risks that could force scope reduction during build.
- Confirm the MVP technology stack (application framework, orchestration layer, model gateway, retrieval store, query execution, logging/tracing, CI/CD and hosting environment).

**Outputs and need level**

| Output | Description | Need level |
|---|---|---|
| MVP scope statement | Users, questions, data assets, tools, answer types and exclusions included in the build. | Mandatory |
| Build backlog | Prioritised implementation items across data, metadata, AI/model use, validation, UI, logging and testing. | Mandatory |
| Acceptance criteria | Minimum evidence required before the MVP can move to formal validation. | Mandatory |
| Known assumptions and risks | Design, data, metadata, access, cost, latency or usability risks that may affect build. | Mandatory |
| Prototype simplifications | Deliberate shortcuts allowed for the current stage, with limits and owners. | Recommended |
| Build timeline and dependencies | Delivery sequence, key dependencies and decision points. | Recommended |

**Watchpoints**

| Watchpoint | Delivery risk | Build response |
|---|---|---|
| Scope expands during build | The MVP becomes too broad to test or govern. | Keep to approved questions and record new ideas in backlog. |
| POC shortcuts enter MVP unchallenged | Weak controls become embedded in the product path. | Label simplifications clearly and decide whether they block validation. |
| Evidence needs are unclear | The MVP may work but still be hard to validate. | Define acceptance criteria before build starts. |
| Exclusions are vague | Stakeholders may assume unsupported questions are included. | Document exclusions and expected refusal or clarification behaviour. |
| Dependencies are unresolved | Build stalls or relies on manual workarounds. | Assign owners and route blockers back to the relevant phase. |

**Practitioner note**

This activity should be straightforward if Phases 1 to 4 were completed properly, but it is still important. A short alignment meeting with product, data, AI, security, evaluation and operating stakeholders should be enough to confirm the MVP scope, technology choices, exclusions and evidence expectations. The build backlog should then be shaped jointly by the product owner and engineering lead so that delivery priorities remain aligned with validation needs.

## 5.2 Confirm MVP governance, ownership and decision rights

**Purpose**: Confirm how the ownership defined in earlier phases applies to the MVP build: who owns each component, who validates outputs, who approves changes and who decides whether defects or limitations block Phase 6 validation[1].

**Activities**

- Confirm named owners for the main MVP components: data access, metadata, retrieval, AI/model routing, prompts, validation, execution, UI, logging, testing and evidence.
- Confirm who validates key outputs, including generated queries, answer quality, caveats, refusals, logs, test results and production-readiness gaps.
- Agree approval points for material changes to scope, prompts, models, tools, validation rules, metadata, access assumptions, supported questions or answer formats.
- Define how defects, failed tests, risks, limitations and production-readiness gaps will be logged, prioritised, accepted or escalated.
- Confirm who owns the Phase 6 validation handover, including evidence quality, unresolved issues and known constraints.
- Identify which governance elements are MVP-ready and which must be strengthened before pilot or production.
- Create or confirm an MVP governance register covering ownership, validation responsibilities, decision rights, issue route, change log, risk acceptance and production-readiness gaps.

**Outputs and need level**

| Output | Description | Need level |
|---|---|---|
| MVP governance register | Ownership, validation responsibilities, decision rights, issue route, change log, risk acceptance and governance gaps. | Mandatory for MVP / Recommended for POC |

| | | |
|---|---|---|
| Component ownership map | Named owners for build components and evidence areas. | Mandatory |
| Approval route | How material changes to scope, prompts, models, tools, validation, metadata or access are approved. | Mandatory |
| Phase 6 handover owner | Named owner accountable for evidence quality and validation handover. | Mandatory |
| Production governance gaps | Governance items still required before pilot or production. | Recommended |

**Watchpoints**

| Watchpoint | Delivery risk | Build response |
|---|---|---|
| Governance is treated as later-phase work | Build decisions become informal and hard to validate. | Define minimum decision rights before implementation starts. |
| Prompt or model changes are undocumented | Test results become difficult to reproduce or trust. | Log prompt, model and routing changes with owner and reason. |
| No one owns validation failures | Issues remain unresolved or are hidden in workarounds. | Assign each failure to a named owner and decision route. |
| Business validation is missing | Answers may look technically correct but be wrong in business terms. | Involve business and semantic owners in answer review. |
| Security assumptions are accepted silently | Access or exposure issues may become embedded in the MVP. | Record assumptions, unresolved risks and approval status. |

**Practitioner note**

This activity is often overlooked because it can feel administrative during build, but it is critical for later validation, pilot and production readiness. Governance creates clear ownership, and clear ownership keeps senior stakeholders engaged when trade-offs, defects, risks and limitations need decisions rather than informal workarounds.

## 5.3 Set up environment, CI/CD and test harness

**Purpose:** Create the technical scaffolding needed to build, test, trace and change the system safely. For a POC this may be minimal; for an MVP it should be strong enough to support validation, user testing and later handover.

**Activities**

- Set up development and test environments with approved access to data, metadata, models and tools.
- Configure version control, deployment route and CI/CD discipline appropriate to the delivery stage.
- Set up secrets, credentials and access controls for model, data and tool connections.
- Create a repeatable test harness for success, failure, unsafe and regression cases.
- Configure logging, tracing, cost and latency capture from the start.

**Outputs and need level**

| Output | Description | Need level |
|---|---|---|
| MVP environments | Controlled development and test environments with approved access. | Mandatory |
| Versioning and deployment route | Controlled route for code, prompts, configuration and deployment. | Mandatory |
| Secrets and access setup | Controlled handling of credentials, keys and tool permissions. | Mandatory |
| Test harness | Repeatable tests for success, failure, unsafe and regression cases. | Mandatory |
| Observability baseline | Logs, traces, cost and latency signals needed for validation. | Mandatory for MVP / Recommended for POC |

**Watchpoints**

| Watchpoint | Delivery risk | Build response |
|---|---|---|
| Local prototype only | The MVP cannot be reproduced, tested or handed over. | Use a controlled environment and documented setup. |
| No prompt or config versioning | Behaviour changes cannot be explained or reproduced. | Version prompts, model settings and configuration. |
| Weak test harness | The MVP works in demos but fails under realistic questions. | Include success, failure, unsafe and regression cases. |
| Logging added too late | Failures cannot be diagnosed or used for validation. | Capture traces from the start of implementation. |
| Secrets handled informally | Keys, credentials or permissions may be exposed. | Use controlled secrets and access management. |

| | | |
|---|---|---|
| Cost and latency ignored | The chosen design may be unusable at validation or pilot scale. | Capture basic cost and response-time signals early. |

**Practitioner note**

Retrofitting version control, deployment discipline, logging, tracing, secrets management, test harnesses and issue tracking later is usually slower and riskier than setting them up early.

Where practical, developers should be able to create a small local or isolated environment through a documented script, configuration or infrastructure template.

## 5.4 Connect approved data, metadata and tools

**Purpose:** Connect the MVP to the approved data assets, metadata sources and tools required to run the Phase 4 flow. The aim is to prove controlled runtime access, not to broaden the data estate.

**Activities**

- Connect the MVP to approved Phase 3 data assets, using scoped read-only access where possible.
- Connect to the metadata sources needed for runtime grounding, including metric, dimension, join, caveat and example artefacts.
- Connect the required tools, including metadata retrieval, query validation, query execution, logging, tracing and feedback capture.
- Confirm that access boundaries, permissions, row/column controls, masking assumptions and blocked assets are enforced or clearly documented.
- Test the connected data, metadata and tools through representative approved questions to confirm they support the intended end-to-end flow

**Outputs and need level**

| Output | Description | Need level |
|---|---|---|
| Approved connection list | Data assets, metadata sources and tools connected to the MVP. | Mandatory |
| Access boundary record | What the MVP can and cannot access, including blocked sources, fields or actions. | Mandatory |
| Metadata availability check | Confirmation that required runtime metadata is available and usable. | Mandatory |
| Connection test results | Evidence that data, metadata and tool connections work for supported questions. | Mandatory |
| Unresolved access or tooling gaps | Issues that may block validation, pilot or production readiness. | Mandatory |

**Watchpoints**

| Watchpoint | Delivery risk | Build response |
|---|---|---|
| Unapproved data is connected | The MVP tests a flow that cannot be governed later. | Connect only approved assets and log exclusions. |
| Metadata is available but not usable | Retrieval or answer generation fails at runtime. | Test metadata against supported questions. |
| Tool permissions are too broad | The MVP can query, expose or execute beyond scope. | Restrict tools and document permissions. |
| Access controls are assumed | Security gaps remain hidden until validation. | Test role, row, column and masking assumptions early. |
| Connection tests are too technical | The system connects but cannot answer real questions. | Test connections through the full question-to-answer flow. |

**Practitioner note**

Access metadata may define the user’s security level, role, region entitlement or permitted dimensions, but it should drive an enforced policy before execution. The MVP should test that generated queries, semantic-layer requests or API calls are constrained by those entitlements before restricted results reach the model or answer layer. Metadata should inform access; policy should enforce it.

## 5.5 Implement metadata retrieval and grounding

**Purpose:** Build the retrieval path that provides the model with the governed context it needs to answer safely. The aim is to test whether approved metrics, dimensions, joins, filters, caveats, examples and access rules can be found and applied at runtime.

**Activities**

- Implement retrieval over the approved metadata sources, including metric, dimension, join, caveat, example and access-control artefacts.
- Define how retrieved context is ranked, filtered, versioned and passed into prompts or tools.
- Test retrieval against supported MVP questions, ambiguous questions and out-of-scope questions.
- Confirm that retrieved context is limited to approved, current and user-permitted artefacts.
- Record retrieval failures, weak matches, missing metadata and grounding issues for remediation or backlog.

**Outputs and need level**

| Output | Description | Need level |
|---|---|---|
| Retrieval implementation | Working retrieval path over approved metadata sources. | Mandatory |
| Grounding rules | Rules for what metadata is retrieved, ranked, filtered and passed to the model. | Mandatory |
| Retrieval test results | Evidence that supported questions retrieve the right context. | Mandatory |
| Metadata gap log | Missing, weak, stale or unusable metadata discovered during testing. | Mandatory |
| Retrieval quality measures | Early view of retrieval accuracy, weak matches, false matches and failure cases. | Recommended |

**Watchpoints**

| Watchpoint | Delivery risk | Build response |
|---|---|---|
| Retrieval finds the wrong context | The model may generate valid but incorrect answers. | Test retrieval against known questions and expected metadata. |
| Too much context is retrieved | Prompts become noisy, costly or inconsistent. | Limit context by scope, ranking, version and relevance. |
| Stale or draft metadata is used | The MVP may apply outdated definitions or caveats. | Retrieve only approved/current artefacts unless explicitly testing drafts. |
| Access rules are not applied to metadata | Users may see or infer restricted sources, fields or definitions. | Filter metadata by user entitlement and approved scope. |
| Missing metadata is hidden by prompts | The model compensates instead of failing safely. | Log gaps and clarify, refuse or route for remediation. |

**Practitioner note**

This activity should be straightforward if Phase 3 produced usable metadata artefacts and Phase 4 defined the retrieval pattern. Weak retrieval can quietly break the MVP by grounding the model in the wrong metric, source, join, filter or caveat.

## 5.6 Implement AI/model routing and clarification

**Purpose:** Implement how the MVP decides which AI/model pattern, rule or tool path should handle each request, and when the system should clarify instead of proceeding.

**Activities**

- Implement routing rules for supported domains, question types, model tasks and tool paths.
- Configure model choices for intent handling, metadata ranking, clarification, query generation, answer generation and safe-failure support.
- Define when deterministic rules should override model judgement, especially for scope, access, validation and refusal decisions.
- Implement clarification behaviour for ambiguous metrics, missing filters, unclear time periods, unsupported dimensions or unsafe requests.
- Log routing decisions, model choices, clarification triggers, fallback paths and unresolved routing issues.

**Outputs and need level**

| Output | Description | Need level |
|---|---|---|
| Routing implementation | Working routing logic for supported domains, questions, models and tools. | Mandatory |
| Model/task map | Which model, rule or deterministic control is used for each task. | Mandatory |
| Clarification rules | Conditions that trigger clarification before query generation or answer generation. | Mandatory |
| Fallback and refusal path | What happens when routing fails, context is missing or the request is unsupported. | Mandatory |

| | | |
|---|---|---|
| Routing and clarification test results | Evidence that routing and clarification work for supported, ambiguous and unsupported questions. | Mandatory |

**Watchpoints**

| Watchpoint | Delivery risk | Build response |
|---|---|---|
| One model handles everything | Cost, latency and failure risk increase unnecessarily. | Route tasks by complexity, risk and evidence. |
| Routing is hidden in prompts | Behaviour becomes hard to test, reproduce or govern. | Make routing rules explicit and logged. |
| Clarification is too weak | The system guesses when it should ask. | Define mandatory clarification triggers. |
| Clarification is too frequent | Users are blocked by unnecessary questions. | Test against realistic MVP questions and refine thresholds. |
| Model choice is not evidenced | The team cannot justify cost, quality or safety trade-offs. | Capture test results, cost and latency by model/task. |

**Practitioner note**

This activity should not become model shopping. Model choice should be task-led: retrieval and ranking may use embeddings, rules or smaller models; complex query generation may justify stronger models; answer generation can often use a medium model if the result and caveats are well grounded.

Routing, access, validation and refusal should remain controlled by explicit rules and governed logic rather than unconstrained model judgement.

## 5.7 Implement query generation, validation and execution

**Purpose:** Implement the path that turns a grounded user question into a bounded, validated and executable query. The aim is not only to generate SQL, but to prove that generated queries stay within approved scope, access rules, cost limits and validation controls.

**Activities**

- Implement query generation using approved metadata, examples, metric logic, dimensions, joins, filters and access context.
- Apply rule-based validation before execution, including approved assets, joins, filters, permissions, row limits, blocked actions and cost thresholds.
- Execute only validated read-only queries through the approved warehouse, semantic layer or API route; editing, write-back and destructive actions should be blocked.
- Apply query limits and guardrails, such as row limits, timeout limits, scan limits, aggregation rules and restricted joins.
- Capture generated queries, validation outcomes, execution status, server-side performance, errors, latency, cost signals and result shape.
- Test supported questions, unsafe questions, ambiguous questions, access-restricted questions and expected validation failures.

**Outputs and need level**

| Output | Description | Need level |
|---|---|---|
| Query generation implementation | Working generation path using approved metadata and examples. | Mandatory |
| Rule-based validation gates | Deterministic checks applied before query execution. | Mandatory |
| Controlled execution path | Approved read-only route with limits, blocked actions and access controls. | Mandatory |
| Query performance evidence | Server-side runtime, timeout, scan, row-count and cost observations. | Mandatory for MVP / Recommended for POC |
| Query and validation test results | Evidence from supported, unsafe, restricted and failed-query cases. | Mandatory |
| Query issue log | Defects, invalid SQL, wrong joins, excessive cost, access failures and fixes. | Mandatory |

**Watchpoints**

| Watchpoint | Delivery risk | Build response |
|---|---|---|
| Valid SQL is treated as correct SQL | The query runs but uses the wrong metric, grain, join or filter. | Validate business logic, not only syntax. |
| Validation is model-led only | Unsafe queries may be approved by the same system that generated them. | Use rule-based checks before execution. |

| | | |
|---|---|---|
| Access filters are added by the model | User entitlements may be applied inconsistently. | Use governed access metadata and enforced policy logic. |
| Query execution is too permissive | The MVP can scan, join, edit or expose more than intended. | Restrict to read-only approved assets, limits and blocked actions. |
| Server performance is ignored | The query may work once but fail under validation or pilot conditions. | Track server runtime, scan volume, row counts, timeouts and cost. |

**Practitioner note**

This activity is where many T2D prototypes look impressive but become unsafe. The model may generate syntactically valid SQL, but that does not mean the query is correct, authorised, affordable or safe to run. For an MVP, query execution should be read-only, rule-validated, bounded by limits and measured for server-side performance before it is trusted for validation or pilot use.

## 5.8 Implement result interpretation and answer generation

**Purpose:** Implement how query results are turned into clear, grounded and caveated answers. The aim is to make the answer useful to the user without losing the source, calculation, assumptions, limitations or safe-failure behaviour.

**Activities**

- Interpret query results using the executed query, result set, metric definitions, filters, caveats and access context.
- Generate answers in the agreed MVP format, including source, period, metric, filters, assumptions and limitations where relevant.
- Apply answer rules for caveats, uncertainty, empty results, partial results, blocked results and failed queries.
- Test answer outputs against evidence rules, including checks for unsupported explanations, causal claims, recommendations, confidence wording and missing caveats.
- Capture answer examples, reviewer comments, answer-quality issues and required prompt/model improvements.

**Outputs and need level**

| Output | Description | Need level |
|---|---|---|
| Answer generation implementation | Working answer path using query results, metadata and caveats. | Mandatory |
| Answer format rules | Required structure for numbers, tables, narrative answers, caveats, refusals and clarifications. | Mandatory |
| Caveat and limitation handling | Rules for when caveats, assumptions, uncertainty or safe-failure wording must be shown. | Mandatory |
| Answer-quality examples | Examples of good, partial, failed, caveated and refused answers. | Mandatory |
| Answer issue log | Incorrect, overstated, unclear, missing-caveat or unsupported-answer issues. | Mandatory |

**Watchpoints**

| Watchpoint | Delivery risk | Build response |
|---|---|---|
| Answers sound better than the evidence | Users may over-trust partial or weak results. | Require caveats, assumptions and source visibility. |
| The model adds unsupported interpretation | The answer may imply causes, advice or certainty not in the data. | Restrict answer generation to result, metadata and approved context. |
| Caveats are dropped | Known limitations disappear from the user-facing answer. | Treat mandatory caveats as required answer content. |
| Empty or partial results are mishandled | The system may present missing data as a real zero or complete answer. | Define specific empty, partial and failed-query behaviours. |
| Confidence wording is unclear | Users may misread technical uncertainty as business certainty. | Use confidence wording only where defined and understandable. |

**Practitioner note**

Answer generation should be tested with questions that tempt the model to overreach, such as “why did revenue fall?”, “what should we do next?” or “are we performing well?”. Unless the retrieved evidence includes approved causal logic, benchmarks or recommendations, the MVP should return the result, caveat the limitation and explain that the available data does not support a cause, judgement or recommendation. This behaviour should be tested and logged, not left to prompt wording alone.

## 5.9 Build lightweight UI and feedback capture

**Purpose:** Build the minimum user-facing interface needed to test the MVP flow, answer presentation and feedback route. The aim is not to design the final product experience, but to make the system usable enough for validation and controlled user testing.

**Activities**

- Build a lightweight interaction shell for asking questions, receiving answers, seeing caveats and reviewing safe-failure responses.
- Display the answer elements needed for trust, including result, source, metric, filters, assumptions, caveats and limitations where relevant.
- Support MVP interaction behaviours, including clarification, refusal, follow-up questions and visible error handling.
- Capture structured feedback, corrections, defects and questions that could not be answered.
- Ensure feedback links back to traces, generated queries, retrieved metadata, model versions and issue tracking.

**Outputs and need level**

| Output | Description | Need level |
|---|---|---|
| Lightweight UI | Simple interface or interaction shell for MVP testing. | Mandatory |
| Answer display pattern | How results, sources, filters, caveats, limitations and safe-failure messages are shown. | Mandatory |
| Feedback capture route | How testers record usefulness, trust, corrections, defects and missing answers. | Mandatory |
| Trace-to-feedback link | Ability to connect feedback to trace ID, query, metadata, model version or issue. | Mandatory for MVP / Recommended for POC |
| UI issue log | Usability, explanation, trust, display or feedback-capture issues. | Recommended |

**Watchpoints**

| Watchpoint | Delivery risk | Build response |
|---|---|---|
| UI becomes the project | Effort shifts from testing the T2D flow to building product polish. | Keep the interface simple and focused on evidence capture. |
| Answers hide caveats | Users may over-trust results. | Make caveats, filters and limitations visible. |
| Feedback is unstructured | Issues cannot be linked to root cause or validation evidence. | Link feedback to traces, queries, metadata and model versions. |
| Follow-ups are unclear | Users may not know whether context was inherited or reset. | Make key inherited filters or context visible where useful. |
| Errors are too technical | Users cannot tell whether to retry, clarify or escalate. | Use clear error, refusal and clarification messages. |

**Practitioner note**

The Phase 5 UI should be designed with the target users, but it should remain deliberately lightweight. The interface should fit the usage context: a web UI or embedded app may suit desk-based users, while mobile or field users may need a different interaction pattern, including voice where appropriate. Security should remain central: the UI must not expose restricted data, hidden prompts, raw traces or query details to users who are not entitled to see them.

## 5.10 Tune, test and package evidence

**Purpose:** Improve the MVP until it is good enough to enter formal validation, and package the evidence needed for Phase 6. The aim is controlled tuning, not informal prompt patching.

**Activities**

- Run the MVP against supported questions, ambiguous questions, unsafe questions, access-restricted questions and expected failure cases.
- Tune versioned prompts, retrieval, model routing, validation rules and answer behaviour based on test evidence.
- Retest changed behaviour and record before/after results, prompt, model, retrieval and validation versions, and remaining defects.
- Prioritise issues as blocking, must-fix, accepted limitation, deferred improvement or production-readiness gap.
- Package the evidence needed for Phase 6, including test results, logs, traces, examples, known limitations, issue backlog and governance decisions.

### Outputs and need level

| Output | Description | Need level |
|---|---|---|
| MVP test results | Evidence from supported, ambiguous, unsafe, restricted and failure cases. | Mandatory |
| Tuning record | Changes to prompts, retrieval, model routing, validation or answer behaviour, with rationale and evidence. | Mandatory |
| Retest evidence | Before/after results showing whether changes improved behaviour or introduced regressions. | Mandatory |
| Issue backlog | Defects, limitations, risks, accepted gaps and production-readiness items. | Mandatory |
| Phase 6 evidence pack | Working MVP evidence required for formal validation planning. | Mandatory |

### Watchpoints

| Watchpoint | Delivery risk | Build response |
|---|---|---|
| Tuning becomes prompt hacking | Behaviour improves in demos but becomes hard to reproduce or govern. | Log changes, versions, rationale and test evidence. |
| Tests only cover success cases | The MVP looks ready but fails on ambiguity, access or unsafe requests. | Include failure, refusal, clarification and restricted-access cases. |
| Fixes are not retested | Improvements may break previously working questions. | Run regression checks after material changes. |
| Issues are hidden to proceed | Phase 6 starts with unknown or unmanaged risks. | Classify issues and record owners, severity and next action. |
| Evidence is incomplete | Validation teams cannot reproduce or trust the MVP behaviour. | Package traces, tests, examples and decisions before handover. |

### Practitioner note

The final Phase 5 activity should make the MVP better, not just document what happened. Tuning is valid when it is controlled, tested and recorded. It becomes dangerous when prompt changes, model switches or validation relaxations are made informally to make a demo pass. Phase 6 should receive an MVP whose behaviour can be reproduced, challenged and validated.

## 6 MVP evidence decision pack

Phase 5 should end with a working MVP and a clear evidence pack. The output should not be a demo narrative; it should show what was built, how it behaves, what was tested, what changed during tuning, and what remains unresolved before validation, pilot or production.

### 6.1 MVP evidence pack

| Output | Purpose |
|---|---|
| Working MVP | Demonstrates that the approved Phase 4 flow can run end-to-end in a bounded way. |
| Implemented flow record | Shows how user questions move through retrieval, model routing, query generation, validation, execution and answer generation. |
| AI/model decision record | Documents where AI is used, which model patterns were selected and where deterministic controls apply. |
| Test results | Shows supported, ambiguous, unsafe, access-restricted and failure cases. |
| Logs and traces | Allows failures, answers, generated queries, retrieved metadata and model calls to be diagnosed. |
| Quality and cost findings | Summarises early answer-quality, latency, server performance and cost observations. |
| Known limitations and issue backlog | Captures defects, accepted limitations, deferred items and production-readiness gaps. |
| Governance record | Confirms owners, decision rights, approval route, change log and Phase 6 handover ownership. |

### 6.2 Evidence pack quality test

The Phase 5 outputs should be tested before handover. A useful evidence pack should be:

| Quality test | Question |
|---|---|
| Complete | Does it show what was built, tested, changed and excluded? |
| Reproducible | Can Phase 6 replay or inspect the same prompts, models, metadata, queries, tests and versions? |
| Diagnostic | Can failures be traced to retrieval, model routing, SQL generation, validation, execution or answer generation? |
| Governed | Are owners, decisions, accepted risks and change records clear? |
| Bounded | Are scope, access, tool limits, simplifications and known gaps explicit? |
| Actionable | Does the backlog clearly separate blockers, validation issues, pilot risks and production-readiness gaps? |

If these tests fail, Phase 5 may have produced a working MVP, but not enough evidence to support formal validation.

# 7 Exit criteria and handover

Phase 5 should close only when the MVP is usable, observable and evidenced enough to enter formal validation. The handover should make clear what was built, what was tested, what remains weak and what must be validated before controlled user testing.

## 7.1 Required exit outputs

The required exit output is the MVP evidence decision pack described in Section 6, completed to the standard required for the delivery stage.

The exit decision should confirm whether the MVP can proceed to Phase 6 validation, proceed with constraints, remediate build, refine design, refine foundation, close the POC or pause. It should also identify the main constraints, accepted limitations and required next actions.

## 7.2 Handover to later phases

| Later phase | What Phase 5 should hand over |
|---|---|
| Phase 6 — Validation, assurance and remediation | Working MVP, evidence pack, test cases, traces, prompt/model versions, generated queries, validation outcomes, answer examples, known limitations and governance decisions. |
| Phase 7 — Controlled pilot | Validated MVP scope, user-facing limitations, feedback route, issue triage process, support assumptions and pilot constraints. |
| Phase 8 — Production readiness and go-live | Production-readiness gaps, hardening backlog, cost and latency evidence, governance gaps, support needs and operating assumptions. |

A POC may close at Phase 5 if its purpose was only structured learning. An MVP intended for user testing, decision support or pilot use should not bypass Phase 6 validation.

## 7.3 Exit decision wording

Suggested wording for the Phase 5 exit decision:

Phase 5 is complete. The MVP has / has not produced enough evidence to proceed to Phase 6 validation. The agreed decision is [proceed / proceed with constraints / remediate build / refine design / refine foundation / close POC / pause]. The main constraints, accepted limitations and required next actions are [summary].

## 7.4 Practitioner note

A working MVP should not be confused with a validated product. Phase 5 proves that the system can be built and tested in a bounded way; Phase 6 proves whether it is accurate, safe and controlled enough to place in front of users.

---

1. *For a POC, this activity may be lightweight. For an MVP intended to enter validation and user testing, governance should be documented clearly enough to support formal testing, controlled change, issue ownership and later production readiness*↩